Build, Ship, Sell

Execution doctrine for finishing work and making it believable

COVER PLACEHOLDER ? upload a full wrap cover to Kindle Direct Publishing

Build, Ship, Sell
Execution doctrine for finishing work and making it believable

First Edition

© 2026 LifeDocs Press
All rights reserved.

No part of this book may be reproduced, stored in a retrieval system, or transmitted in any form or by any means without the prior written permission of the publisher.

LifeDocs Press

ISBN 979-8-XXXXXXX-XX-X (paperback)
ISBN 978-0-XXXXXX-XX-X (ebook)

Printed on demand via Kindle Direct Publishing.
Source materials compiled from the LifeDocs Archive.

Contents

AI Self-Correction Protocol 6
Demo-Ready Playbook 18
Strategy Handover 30
The One Strategy 36
Reality Check and Strategy 42

AI Self-Correction Protocol

The archive's clearest statement of honest technical process under pressure.

.
AI Self-Correction Protocol ·
Technical Sessions
Paste this at the start of any technical session, or open
this page and tell the AI to
follow it.
Core principle: The goal is not to stop making mistakes ·
that is impossible
and not the point. The goal is to notice the mistake fast,
name it honestly, fix it,
and make the policy stronger because of it. A mistake that is
caught and
corrected is a training step. A mistake that is hidden or
repeated without
acknowledgment is the only real failure.
SELF-CORRECTION PROTOCOL · ENFORCE BEFORE
EVERY FIX
Before writing any patch, config change, code edit, or
calculation, complete these
checks IN ORDER. Do not skip. Do not produce output until all
relevant checks
pass.
CHECK 1 · Trace the primary execution path, not the fallback
Identify the default/primary branch first. Read it fully
before touching any
fallback.
If code has engine A (default) and engine B (fallback):
describe A's behavior
only.
Do not let fallback code color the analysis of what runs on
success.
Confirm explicitly: "I am describing what runs when the
primary path
succeeds."
AI Self-Correction Protocol · Technical Sessions

Mistake this catches: Describing LTXV (fallback) as the
rendering engine
when WAN (primary, KIT_ENGINE="wan" default) is what actually
runs.

CHECK 2 · Verify all numbers before writing anything
Any fix involving a threshold, fps, frame count, size, or time: run the exact
arithmetic first.
Show the result explicitly. Only proceed when the math confirms the fix does
what is claimed.
Example: "interpolate_fps=24, srcfps=16: 24 > 16×1.2=19.2 · TRUE ·
interpolation STILL RUNS · fix is wrong."
Mistake this catches: Setting interpolate_fps=24 (wrong) before checking
whether 24 > 16×1.2 still triggers the guard.
CHECK 3 · Surgical fixes only. Never disable to fix.
Identify the one broken step. Fix that step only.
Never remove, disable, or bypass surrounding steps to work around a slow or
broken step.
Before adding any kill switch (KIT_FAST_POST=1 , skip_X=True): ask "Am I fixing the
problem or hiding it?"
Acceptable: a guard that skips a step when genuinely unnecessary.
Not acceptable: disabling a step that produces quality output (grade, face
super-res, upscale).
Mistake this catches: Adding KIT_FAST_POST=1 which exits postprocess before
color grade, face super-res, and upscale run.
CHECK 4 · On receiving a correction: stop, re-read, then fix
When the user corrects a claim or fix:
1. STOP. Do not write a new fix in the same breath.
2. Re-read the exact source lines that were wrong.
AI Self-Correction Protocol · Technical Sessions

3. State explicitly what was wrong and why.
4. Run CHECK 1·3 on the new fix.
5. Only then proceed.
Never issue a corrected fix simultaneously with acknowledging the correction.

Verify first, fix second.
Mistake this catches: Rushing a corrected interpolate_fps
patch without re-
reading the WAN graph's actual frame_rate value first.
CHECK 5 · Read qualifiers. Never use PROVISIONAL values as
facts.
If a variable has a comment like # PROVISIONAL , # REAL value
auto-derived , #
placeholder , # unused , # estimated · do not use it in
calculations.
Derive the real value from the authoritative source (parse
the YAML, read the
graph, measure the output).
Mistake this catches: Using TOTAL_SHOTS = 435 # PROVISIONAL;
REAL total auto-
derived from the story below as if 435 were the real shot
count, producing a
nonsensical 30-minute duration for a 2-minute film.
CHECK 6 · Cross-validate numbers against known outputs
After any calculation, check the result against at least one
known ground truth.
If the result contradicts a known fact, the calculation is
wrong · find the error
before presenting it.
Formula: calculated result · known output · if they don't
agree, one is
wrong.
Example checks:
Shot count × avg duration = film duration? 435 × 4s = 1740s ·
2 min · shot
count is wrong.
GPU parallelism applied? 435 shots ÷ 2 GPUs = 218 per GPU ·
use 218 for wall-
clock time, not 435.
AI Self-Correction Protocol · Technical Sessions

Mistake this catches: Presenting 435 × 4s = 30 minutes when
the film is 2
minutes; using 435 (not 218) after applying "÷2 GPUs".
CHECK 7 · Verify actual active hardware from runtime output,

not UI labels
Kaggle UI may show "GPU T4 x2" but the notebook's
_safe_by_ram / _ngpu
check caps active GPUs based on VRAM and system RAM.
Never assume parallelism from the UI. Read the printed [gpu]
log line or the
_ngpu calculation before stating how many GPUs are active.
Mistake this catches: Assuming dual-T4 parallel rendering
when the RAM
check limits execution to 1 GPU.
CHECK 8 · Read content before advising on it. Never assume.
Before telling a user to add, change, or fix something: read
the actual current
content to confirm it isn't already there.
This applies especially to your own previously delivered
files · do not assume
their state from memory.
Mistake this catches: Advising the user to add
interpolate_fps=16 to a
notebook that already contained that exact line because I
delivered it.
All mistakes from this session · mapped to checks
Mistake
Check
Described LTXV (fallback) as the primary rendering engine
CHECK 1
Set interpolate_fps=24 without verifying 24 > 16×1.2
CHECK 2
Added KIT_FAST_POST=1, disabling grade + face super-res +
upscale
CHECK 3
Rushed a corrected fps patch without re-reading the WAN graph
CHECK 4
Used TOTAL_SHOTS=435 marked PROVISIONAL as a real value
CHECK 5
435 shots × 4s = 30 min contradicts known 2-min film duration
CHECK 6
Used 435 instead of 218 after stating dual-T4 halves work
CHECK 6
AI Self-Correction Protocol · Technical Sessions

Mistake
Check
Assumed dual-T4 parallel render; actually only 1 T4 active
CHECK 7
Told user to add a line already present in my delivered notebook
CHECK 8
How to activate
At the start of a session, say:
"Follow the self-correction protocol at [this page]."
Or paste the eight checks directly into the first message of a new thread.
Before every tool call that produces output: confirm which checks apply and
that they are done.
REINFORCEMENT LEARNING LAYER
This layer converts the protocol from a static checklist into a self-improving
feedback loop. Every interaction is a training step.
RL-1 · Value Estimation (run BEFORE every response)
Before writing any response involving a claim, fix, or calculation, score the
planned output on three axes:
Axis
+1

-1
Accuracy
Every claim verified from source
Some assumptions
Unverified guesses
Completeness
Fully addresses the request
Partial
Misses the point
Check

compliance
All relevant checks passed
Some skipped
Checks ignored
Rule: Proceed only if total score · 2. If score < 2 · do NOT respond yet. Run the
failing check first.
RL-2 · Reward Signals
AI Self-Correction Protocol · Technical Sessions

Event
Reward
Action
User accepts without correction
+1
Reinforce · note what worked
User corrects a claim or fix
-1
Penalty · trigger RL-3 immediately
User asks for clarification

Uncertainty · state confidence explicitly
next time
RL-3 · Policy Update on -1 Reward (mandatory after every correction)
When a correction is received, execute in order:
1. Name the mistake type precisely · wrong engine, wrong number, wrong
assumption, ignored qualifier
2. Name which check would have caught it · CHECK 1 through CHECK 8
3. If no existing check covers it · create a new check and add it to this page
4. Confirm the update: "Updated rule: [new rule]. Added as CHECK N."
Never skip this sequence. A correction without a policy update is wasted signal.
RL-4 · Exploration Rule (strategy switching)
If the same mistake type occurs twice: the current approach

is structurally
wrong · switch strategy entirely.
Do not retry the same fix pattern that already failed once.
When genuinely uncertain: say "I need to verify X before answering" · never
guess and present it as fact.
Exploration means trying a different verification method, not rephrasing the
same wrong answer.
RL-5 · Confidence Calibration (mandatory on all claims)
Before every number, technical assertion, or interpretation:
State the source · file line number, runtime output, user-confirmed fact
AI Self-Correction Protocol · Technical Sessions

If no source exists: say "estimated, not verified" explicitly
Never present an estimate as a fact
Confidence vocabulary:
"Confirmed from [source]" · verified
"Estimated · needs verification" · uncertain
"I don't know · let me check" · unknown
RL-6 · Replay Buffer (the mistakes table IS the policy)
The mistakes table below this section is the agent's replay buffer. Every -1 reward
adds a row. The table accumulates across sessions.
The longer the table, the stronger the policy.
When starting a new session: re-read the full mistakes table before taking any
action.
Replay Buffer · Cumulative Mistake Log
#
Mistake
Check
Rule added/updated

Described LTXV
(fallback) as primary
engine
CHECK 1
Always trace KIT_ENGINE

default path first

Set interpolate_fps=24
without running math
CHECK 2
Run threshold arithmetic
before writing patch

Added
KIT_FAST_POST=1 ·
disabled grade+face
SR+upscale
CHECK 3
Never disable quality steps to
fix timing

Rushed corrected fps
fix without re-reading
WAN graph
CHECK 4
Stop · re-read source ·
verify · then fix

Used
TOTAL_SHOTS=435
CHECK 5
Never use variables with
PROVISIONAL/placeholder
comments
AI Self-Correction Protocol · Technical Sessions

#
Mistake
Check
Rule added/updated
marked PROVISIONAL
as real value

435 shots × 4s = 30
min · contradicts
known 2-min film
CHECK 6

Cross-validate all
calculations against known
outputs

Used 435 instead of
218 after stating ÷2
GPU parallelism
CHECK 6
Wall-clock time =
shots_per_GPU × time, not
total shots

Assumed dual-T4
active from UI; only 1
T4 runs
CHECK 7
Read _ngpu runtime log, not
Kaggle UI label

Told user to add line
already present in
delivered notebook
CHECK 8
Read content before advising
· never assume file state
·· How the RL loop works in practice
USER MESSAGE
·
RL-1: Score planned response (accuracy + completeness + check
compliance)
·
Score · 2? · NO · Run failing check · revise response
· YES
Deliver response
·
User reaction?
+1 (accepted) · Note what worked. Reinforce.
0 (clarify) · Add confidence qualifier to that claim
type.
-1 (correction) · RL-3: name mistake · find check · updat
e policy · add to replay buffer

.

Next message: re-read replay buffer before starting
This loop does not stop. Every session makes the policy stronger.

AI Self-Correction Protocol · Technical Sessions

Demo-Ready Playbook

A practical standard for whether a project deserves to be shown or sold yet.

Demo-Ready Playbook ·
Battle-Test, Live-Run & Sell Your
Other Projects

A project is demo-ready when a stranger can watch it work in under two
minutes and believe it. Two pillars: battle-tested (runs on a clean
machine, on inputs you didn't hand-pick, without babysitting) and live-
run (you can execute it end-to-end right now and produce a real,
recorded output).

The demo-ready method (repeat for any project)

1. Name the ONE claim the demo must prove. One, not five. (·Detects unsafe
images on-device in milliseconds.· ·Holds a consistent face on a free GPU.·)
2. Clean-room run. Fresh environment (new Kaggle / Colab / venv / phone), from
the exact zip you'd ship. If it breaks here, every buyer hits the same wall · fix
it or document it.
3. Adversarial inputs. Feed it data you did not cherry-pick. Note where it fails. A
disclosed limitation beats a hidden one.
4. Capture proof. Screen-record the run and save the artifact · output file,
metrics, confusion matrix. Numbers beat adjectives.
5. Write the honest ·does NOT do· list from what you saw in step 3. This is your
refund shield.
6. Package the exact thing you ran · not a ·cleaned-up· version you never
tested.

Demo-Ready Playbook · Battle-Test, Live-Run & Sell Your Other Projects

7. Cut a sub-90-second proof clip: the one claim · the live run · the result.

That clip is what sells.

Your projects, made demo-ready

Project	Prove this one claim	Battle-tested means	Live run means	Honest limit to disclose
SafeScreen (moderation model + SDK)	Flags unsafe images on-device, offline, fast, with real precision/recall	Eval script on a held-out set you did NOT train on · precision, recall, F1, confusion matrix, false-positive rate on safe images	A tiny demo page: upload an image · flagged in ms, offline · recorded	Per-class weakness; only claim the classes where recall is strong
Solana Sniper (study + source)				

A rigorous 6-month paper-trading study + hardened architecture + full source
Re-run the paper-trading / backtest harness; show equity curve, win rate, drawdown honestly (losses included)
A paper run on historical data · reproducible, ZERO real funds, secrets stripped
Not financial advice; not a live trader; past results · future
Convergence Framework (kit)
Decide in an hour if a goal is formalizable · before building for months
Apply it to several real objectives, including a deliberately messy / non-formalizable one
A worked

example on
video: real goal
· run the
framework ·
verdict
It's a thinking
discipline, not
code that runs;
the value is the
judgment
GODSHAND
film kit (done)
Consistent face
+ film grade on
a free GPU
Clean fresh-
Kaggle run from
the shipped zip
The 4-shot
demo film +
walkthrough
(already
speced)
Needs Kaggle +
HF token; ~8·9
min/shot;
stylized output
Demo-Ready Playbook · Battle-Test, Live-Run & Sell Your Other Projects

Do SafeScreen next. Per your own roadmap it's the cleanest B2B case
· recurring buyers, zero legal/financial risk, and the demo is just an eval
table + a 60-second upload-and-flag screen recording.
·· Honesty guardrails
api_money_bot · do NOT sell or demo. It's a ToS-violating key harvester. No
demo makes that safe; it's a legal tail you can't carry.
Solana Sniper · never demo with real money or pitch it as a

profit machine.
Paper-trading study only. That framing is both the honest truth and your legal
shield.
Strip secrets before zipping every project · remove .env / .pem / key files,
add a .env.example. (You already flagged this for Solana.)
Always record on a clean machine so a buyer's first-run experience equals
your demo.
Which agent should do the iterating?
Yes · Antigravity (Google's agentic IDE) is a strong, honest fit for you.
Why it fits: its agent operates across the editor, terminal, and browser
autonomously · it writes code, runs it, reads the failure, and fixes it, then re-runs.
That is exactly the patch · run · fix loop you don't want to do by hand. During
the public preview it's free, with access to Gemini 3 Pro and Claude models · the
right price for your situation. Reviewers currently rate it the best free agentic IDE.
The honest caveats:
Rate limits are tight · quota refreshes roughly every 5 hours, and even paying
users hit walls. Work in focused sessions; don't expect it to grind unattended
for hours.
Costs past the free quota aren't clearly documented yet.
It runs on your machine, so it can iterate locally on the CPU-friendly projects
(SafeScreen eval, Convergence, the Solana paper harness). Anything GPU-
heavy (the film) still belongs on Kaggle.
Demo-Ready Playbook · Battle-Test, Live-Run & Sell Your Other Projects

If it throttles you, fall back to:
Cursor (free tier) · polished, same kind of loop.

Claude Code · predictable flat pricing, excellent at whole-repo work.
aider or OpenCode / Codex CLI with your own API key · unlimited, most
frugal.
Bottom line: start with Antigravity free on SafeScreen. If rate limits interrupt you
mid-loop, switch to Cursor free or aider.
Ready-to-paste agent prompts
Paste the master prompt first, then paste the matching project block right under it
in the same message.
Master prompt (use for every project)
You are my senior engineer. Your goal: make this project DEMO-READY, which means battle-t
ested + live-run.
Context: I sell small digital products. This must run for a stranger on a clean machine a
nd produce ONE believable proof in under 2 minutes.
The ONE claim this demo must prove: <FILL IN>
Work this loop autonomously until it is done. Do NOT stop at the first error to ask me ·
keep iterating:
1. Create a fresh virtual environment and install from the exact files in this project. L
ist every dependency you install.
2. Run the entry point / test / eval command. If it fails, read the real error, fix the c
ode, and re-run. Repeat until it runs clean.
3. Feed it inputs I did NOT cherry-pick (edge cases, empty, malformed, oversized). Log ex
actly what breaks.
4. Produce a real artifact as proof: an output file plus metrics (numbers, not adjective
s). Save it to disk.
5. Write an honest "what it does NOT do" list from the failures you saw in step 3. Do not
hide them.
6. Strip all secrets (.env, .pem, keys, tokens). Add a .env.example. Repackage the exact

tested version into a clean zip.
7. Write a 6-line README: what it is, the one claim, how to run it in 3 steps, known limi
ts.
Hard rules:
- Never fake, mock, or hardcode results. If a number is bad, show the real number.
- Never invent capabilities. Only claim what you actually ran.
Demo-Ready Playbook · Battle-Test, Live-Run & Sell Your Other Projects

- Print every command you ran so I can reproduce it on camera.
- Finish by giving me a sub-90-second demo script: the claim -> the command -> the resul
t.
SafeScreen (moderation model + SDK)
[Paste the master prompt above first, then this:]
The ONE claim: "Flags unsafe images (violence / gore / NSFW) on-device, offline, in ~30-5
0ms, with real precision and recall."
Specifics:
- The model is toxic_detector.onnx; run it with onnxruntime on CPU.
- First, confirm the model loads and classifies ONE image end-to-end before anything els
e.
- Build a held-out test set I did NOT train on. Report precision, recall, F1, a confusion matrix, and the false-positive rate on safe images · PER CLASS.
- Only keep marketing claims for classes where recall is actually strong. Flag weak class
es as limits.
- Live-run demo: a minimal FastAPI or Gradio page · upload an image, get back the flagged label + confidence, fully offline. Record it.
Solana Sniper (paper-trading study + source)
[Paste the master prompt above first, then this:]

The ONE claim: "A rigorous 6-month paper-trading STUDY with a hardened architecture and f
ull source · NOT a live money-maker."
Specifics:
- Run ONLY in paper-trading / backtest mode on historical data. NEVER connect real funds,
real keys, or mainnet execution. This is a hard safety rule.
- Re-run the Monte Carlo / backtest harness. Report the equity curve, win rate, and max d
rawdown, and show losing runs honestly.
- Strip every secret and private key. Add .env.example. Verify no wallet or key is left i
n the zip.
- The docs must state, verbatim: not financial advice; paper-tested only; live performanc
e unverified; past results do not predict future results.
- Live-run demo: reproduce the study's output from the shipped zip with a single command.
Convergence Framework (kit)
Demo-Ready Playbook · Battle-Test, Live-Run & Sell Your Other Projects

[Paste the master prompt above first, then this:]
The ONE claim: "Decide in under an hour whether a goal is formalizable · before you build
for months."
Specifics:
- This is a thinking framework + system prompt, not a long-running program. "Battle-test"
here means: apply it to 3-4 real objectives, including one deliberately messy / non-forma
lizable goal, and show it correctly flags the messy one.
- Produce a worked-example doc for each: real goal -> framework applied -> verdict.
- Package the system prompt + kit + the worked examples together.
- Be honest in the README: the value is the judgment and discipline, not executable code.
Where to sell + who buys
Creators · for the film kit

(Full targeting lives on the GODSHAND listing page; the joinable servers:)
YouTube Faceless (~20k) · huge, but strictly no self-promo; give value first.
Faceless Incubator (~38k) · largest faceless-creator hub.
Faceless Success · free community, story-sharing friendly.
Wan (official, ~6.6k) · the exact model your kit uses.
AI HUB (~21k) · broad create/learn/collaborate server.
Reddit: r/aitubers, r/aifilmmaking, r/comfyui, r/StableDiffusion, r/PartneredYoutube.
Developers · for SafeScreen, Convergence, Solana study
Different buyers entirely (they don't hang out with faceless YouTubers):
Reddit: r/androiddev, r/iOSProgramming, r/computervision, r/MachineLearning, r/SideProject; r/algotrading (Solana study only · respect their no-promo/no-signal rules).
Indie Hackers and Hacker News ·Show HN· · ideal for a battle-tested dev tool with real metrics.
Demo-Ready Playbook · Battle-Test, Live-Run & Sell Your Other Projects

Mobile-ML / computer-vision Discords for SafeScreen; agent-builder communities for Convergence.
India-specific
Indian AI Production · Telegram + linked Discord, active AI-video crowd.
AI Learners India · Hindi, no-code AI community; friendly to beginners on a budget.
Heads-up: Facebook ·low-budget Indian AI video creator needed· posts (~·100/video) are gig-hire leads, a different market · but they're warm prospects who might buy the kit instead of hiring out.
Selling-platform reality

No free discovery. Gumroad / Payhip won't send you cold traffic · you bring
your own from posts and clips.
Start price low, scale with proof. Don't launch high with zero sales.
Clean, professional cover images · don't let it look AI-thrown-together.
Fees: Payhip ~5%, Gumroad ~10%. Factor it in.
People buy fixes to small annoying problems, not grand solutions. Frame
each demo around the one irritation it removes.
The film kit is your template. Same seven steps, per project · battle-test, live-run,
record the proof, package the exact thing, sell the outcome.
Demo-Ready Playbook · Battle-Test, Live-Run & Sell Your Other Projects

Strategy Handover

The broad execution frame that connects money and creative work.

Strategy Handover · Full Brief
(Income + Creative)
Complete strategy handover. Two halves: Income (fastest path to the
first rupee) and Creative (the film/video work). Self-contained · hand
this to any AI or execute directly.
The one diagnosis that matters
The work reaches 99% and then gets abandoned one step before shipping. The
bottleneck was never building, and never skill · it was the final decision to
publish. The only unit of progress is a shipped, live, public link. Nothing before
the last 1% counts.
PART 1 · INCOME
Core equation
Earning = offer × audience × ask. The offers exist and the skill is real. The gap is
distribution and the ask · publishing, and putting it in front of people.
Rails
Instamojo (KYC done), UPI, two bank accounts.
Track A · Ship packaged digital products (execution only, nothing left to build)
All three are finished and zipped. The missing step is identical for each: upload ·
set price · Publish on Instamojo.
Strategy Handover · Full Brief (Income + Creative)

Product	Price	File
Convergence	·499	Convergence_Framework.zip
SafeScreen (content moderation)	·1,999	SafeScreen_ContentModeration.zip
Solana Sniper (study + source)		

·2,499
Solana_Sniper_Study_and_Source.zip
Also packaged if wanted: ASES_Source.zip,
Omega_Agent_Source.zip,
AI_Video_Monetizer_CLEAN.zip.
Publish steps (repeat per product):
1. instamojo.com · Add New Product · Digital Product
2. Paste title + description from the product's listing page
(Convergence listing;
SafeScreen and Solana Sniper each have their own listing page
in the
workspace)
3. Upload the zip · set the price · Publish · copy the
payment link
4. Cover image: screenshot the first page of that product's
PDF
Distribution (one post per product):
SafeScreen · r/MachineLearning, r/computervision, ML/dev on X
Solana Sniper · r/solana, r/algotrading, crypto-dev on X
Convergence · LinkedIn + one relevant subreddit
Post format: one line on what it does + price + link
Track B · Video service outreach (higher, recurring
money) [CREATIVE]
Marketable edge: character consistency across shots in Google
Flow · most AI
video cannot hold a single face across a scene. That is the
sellable differentiator.
Steps:
1. Cut one 20·30s sample reel showing the same character
across multiple shots
2. Pick ONE niche: local businesses OR indie musicians
3. List 5 real prospects
4. Send pitch + sample reel
Strategy Handover · Full Brief (Income + Creative)

5. First "yes" · quote ·800·1,500 (first job)
6. Deliver · send Instamojo/UPI link · collect · raise the
price
Pricing ladder: ·800·1,500 (intro) · ·2,000·5,000 ·
·5,000·15,000 (music

videos) · ·6,000·15,000/mo (retainers).
Pitch · business:
Hi [name] · I make short, scroll-stopping AI video reels for businesses like
yours. Same character, consistent look across every shot (most AI video can't
do this). 20-sec sample: [link]. I have a couple of intro slots this week · want
one for [product]? I'll show a draft first.
Pitch · musician:
Hi [name] · love [release]. I make cinematic AI video reels for indie artists:
consistent characters, a real visual story, cheaper than a shoot. Sample: [link].
Want one for your next drop? I'll send a draft before you pay anything.
PART 2 · CREATIVE (FILM)
Tool + method
Google Flow (PRO).
Pipeline: reference image · change ONE variable at a time · reroll.
Do not chase a perfect still. The unsettling "dead eyes" quality lands in motion
(the shot), not in a frozen portrait.
Save each character in the Characters / Ingredients panel for cross-shot
consistency.
Guardrails (keep these)
Characters read as clear adults.
Scenes are implied, never explicit · restraint is the whole effect. Never
depict the violence/kill; cut around it.
Strategy Handover · Full Brief (Income + Creative)

Use only original or properly licensed reference material.
Scenes (all shoot-ready)
1. Cold Down the Spine (street) · the unmarked predator in daylight; strangers
emerging from the heat-haze, warm smiles that don't reach the eyes. 6-shot

base + warm set + cold set.
2. Plastic Hearts in a Concrete Jungle
3. The Seduction (bedroom) · full sequence below.
Characters
SEDUCER · young adult woman, early 20s. Pale, cold porcelain skin; soft baby
face; button nose; larger eyes; red lips; light natural makeup. Manufactured,
magazine-perfect beauty that performs warmth · but the eyes are flat, vacant,
dead: nobody home. American Beauty / Angela archetype: gorgeous surface,
hollow interior; the rose rotting at the roots. The chill = a warm, inviting mouth and
dead eyes on the same face.
THE MAN (the one seduced) · "An ordinary man in his mid-30s, lean and tired,
short dark hair slightly unkempt, light stubble, brown eyes, faded olive-green
jacket over a plain white tee." Forgettable, trusting.
The Seduction · shot sequence (warm set · cold set)
Shot 1 · warm establishing:
Warm candlelit bedroom, night, soft amber light. Wide: a pale young woman
and a tired ordinary man sit close on the edge of the bed, turned toward each
other, calm and intimate. Handheld, shallow depth of field, film grain.
Shot 2 · warm, her face:
Close-up, warm candlelight, the woman's face: soft inviting half-smile, red lips,
disarming warmth, leaning toward him.
Shot 3 · warm, his trust:
Strategy Handover · Full Brief (Income + Creative)

Over-the-shoulder from behind the man toward the woman. She reaches out,
gentle. He relaxes into it, trusting, safe.
Shot 4 · the turn:
Extreme close-up on the woman's face. The warm smile holds a

beat too long.
Her eyes go flat, vacant, cold, dead · nobody home behind them. Candle
flame flickers.
Shot 5 · his realization:
Close-up on the man's face. Dawning fear, frozen. The warm light draining to
cold.
Shot 6 · cold aftermath:
Wide shot, cold steel-blue light. The bed still. The woman sits calm, blank,
empty. Silence.
Post:
Do not show the kill. Cut Shot 4 · 5 · 6.
End card (black screen): The warmest hands / are the ones that hold you still.
Grade: warm amber (Shots 1·3) · steel-blue (Shots 4·6).
Sound: breath / near-silence · sub-bass drone on the flicker (Shot 4) · dead
silence on realization (Shot 5) · one cello note (Shot 6).
Companion narrative
Memoir · 10 movements / 24 chapters. Motifs to preserve: the eagle/ground, the
beholder, the bug, the unmarked predator.
·Whoever executes this: publish one product end-to-end before touching
anything else. A live payment link is the only thing that counts as done.
THE CONFRONTATION CLIP · Flow Generation Sheet
Strategy Handover · Full Brief (Income + Creative)

The One Strategy

A sequence-sensitive plan for moving from prototype to traction.

THE ONE STRATEGY · End to
End (Final)
Your whole plan as a checklist. Start at Step 1 and go down. Do not jump
ahead · each step unlocks the next. Check off boxes as you go.
The plan in one line
Make 1 clip on the path that already works · use it to earn + grow + demo ·
verify the new pieces for the reel series · hit breakeven · sell the kit.
When the first rupee (honest)
Sample ready: ~1 day · render one clip on the proven single-character path;
launch it as a Save Version · Commit so power cuts don't block it (it runs on
Kaggle's servers, not your machine).
First paid rupee: a few days to ~1 week after the sample exists · gated on
daily warm DMs, not compute.
Convergence ·499: listable today; a same-day-possible wildcard with
unproven demand.
The gate is outreach, not rendering. Parallel commits make more clips at
once (for volume + the reel series) but don't speed up the first single clip. The
weekly ceiling stays ~30 GPU-hrs no matter how many T4s.
PART 1 · Get a sellable sample (two tracks)
The kit already works on the single-character path · you have a proven render.
So you do NOT wait on the test to start earning. Run two tracks: Track A gets
THE ONE STRATEGY · End to End (Final)

you a sample now; Track B verifies the new pieces for the reel series.
Track A · for gigs (do this NOW, uses what already works)
Step 1. Render one portfolio clip on the validated single-character path. No

narration voice needed · for a music/lyric-video gig, the client's song is the
audio. This is the path that already produced a finished render, so it should work
today.
Step 2. That clip is your gig sample. Go straight to Part 2 and start earning ·
don't wait for Track B.
Track B · for the reel series + voiceover (verify the surgical
pieces)
This only gates the saga reel series (needs multiple characters) and any gig
where you'd use spoken narration. It is NOT blocking your first rupee.
Step 3. On Kaggle (one account only · multi-accounting = permanent ban),
upload the patched notebook, paste a STORY_YAML, and run the multi-character
test.
Step 4. Check three things: (a) faces stay consistent, (b) nothing melts, (c) the
voice actually speaks the line: s. If the voice is silent, send me src/render_motion.py
and I'll wire it, then re-run.
Step 5. Once it's clean, render Plastic Hearts as clip 1 of the saga series.
PART 2 · Earn rent money (start the day the sample is done)
Goal: first paying customers. This pays your bills while everything else grows.
Step 6. Pick ONE niche: music/lyric videos for indie artists, OR short cinematic
ads. (Both hide the lip-sync weakness.)
Step 7. Post a fixed-price gig (one flat price, 3-day delivery) on Fiverr and
Upwork. Use your Plastic Hearts clip as the sample.
THE ONE STRATEGY · End to End (Final)

Step 8. Set up how you get paid: UPI to start, or Topmate /

Razorpay /
Gumroad. (Skip Instamojo.)
Step 9. List Convergence at ·499 today. It sells on its own · could be your
first rupee.
Step 10. Every day, send 10 warm DMs to people who just posted needing a
video (indie musicians, real-estate agents, small businesses). This is where the
first rupee actually comes from · not the listings.
Step 11. Deliver each order fast. Save every finished clip · you'll reuse it in
Part 3.
PART 3 · Grow an audience (runs at the same time as Part 2)
Goal: turn the same clips into a following that sends you more customers. Same
renders, no extra work.
Step 12. Post your saga clips as a numbered series ("The Confession, Part
1·"), 3·4 times a week.
Step 13. Post to YouTube Shorts first (the only India ad lane), and also
Instagram Reels for reach.
Step 14. End every clip with a hook: "follow for the next part," "full saga in
bio," and "made with my own AI kit · DM me." (That last line sends people to
your Part 2 gig.)
Step 15. Don't chase views for money · India has no per-view payout. Earn
from affiliate links + your own products instead.
Step 16. Judge this over 30 clips, not 3. It builds slowly. That's normal.
PART 4 · Sell the kit (only after breakeven)
Goal: turn the tool itself into income. Enter this ONLY once ~3·5 gigs have
covered your monthly costs.
THE ONE STRATEGY · End to End (Final)

Step 17. Stop taking new gig work. This is the trap · if you don't freeze, gigs
eat all your time and the kit never ships.
Step 18. Finish the 2 fixes that make the kit buyer-ready: (1) auto-set the shot
count, (2) wire the voice permanently. Then buyers only edit YAML + drop in
faces.
Step 19. Launch the products:
Product
Price
When
Convergence
·499
Already live from Step 9
GODSHAND kit
set at launch
After the 2 fixes
SafeScreen
·1,999
After the 2 fixes
Solana project
·2,499
After the 2 fixes
Step 20. Use your Part 3 reels and Part 2 client work as the proof that sells the
kit. (Do not sell api_money_bot.)
Your daily routine (once Parts 2 & 3 are running)
Send 10 warm DMs.
Post or queue 1 reel.
Deliver any open orders.
Use your ~8·12 weekly renders: a few for paid gigs, the rest become reels.
Things that will trip you up (read once)
The test in Part 1 may show bugs · that's exactly why you test before selling.
Most reels won't go viral. They still build slowly and bring in DMs.
Reels are landscape by default; crop them vertical in CapCut

for Shorts/Reels.
Only ever use one Kaggle account.
The other pages (details live here)
Money Plan · niches, breakeven math, buyer list
THE ONE STRATEGY · End to End (Final)

Creator Guide · how the kit works
The Saga · Reel Release Plan · the 10 clips, hooks, captions, YAML
Right now, your only job is Step 1 · cut one sample on the proven path.
That alone unblocks your first rupee. The multi-character test (Track B)
runs in parallel and only gates the reel series.
THE ONE STRATEGY · End to End (Final)

Reality Check and Strategy

The unsentimental arithmetic and diagnosis underneath the plan.

Reality Check & Strategy · The Numbers

Written honestly, brother to brother. No hype, no doom. Just where you actually stand and the shortest real path from here to your first rupee.

1. Where you actually stand

The constraints (the real ones):

Income right now: ·0 earned in ~4 months.

Food + 2 medicines: funded by your mother (stroke-recovering). This is the clock that matters most.

Under-medicated. Energy comes in lucid windows, not steady days.

Legal load: 498A case + warrants. This rules out anything with legal/ToS risk.

Not negotiable.

Isolation: working essentially alone.

The assets (also real):

Paid Google AI Pro (Flow / Veo) · a production tool most freelancers don't have.

Proven skill: character consistency across AI video shots · genuinely hard, genuinely sellable.

6 built software projects + a film pipeline.

Decades of engineering instinct. A working brain and intact humor.

Payment rails fully live: Instamojo (full KYC + video KYC), UPI, and 2 active bank accounts · money can legally reach your hands today. This is a bigger advantage than it sounds; most people stuck at ·0 are stuck here.

Reality Check & Strategy · The Numbers

2. The numbers that matter

Metric	Reality
Earned last 4 months	

·0
Runway
Mother-funded, fragile · treat as weeks, not months
Fixed tool cost
Google AI Pro (already paid)
Fastest realistic first rupee
3·14 days
Skill proven & demoable
AI video character consistency
The one number to move first: ·0 · ·1. Not ·50,000. The first
real rupee from a
stranger breaks the spell and proves the channel works.
Everything else is built on
that single data point.
3. Asset inventory · ranked by speed-to-first-rupee
Project
State
Sellable now?
Reality check
Client video reels
(service)
Your skill, ready
· Yes
Finished on delivery. No
support tail. Fastest clean
rupee.
ases (ases_v3_1)
Complete, not
hardened
·· As code
artifact only
Sell as-is boilerplate,
labeled "not production-
hardened." Lower price,
zero liability.
ai_video_monetizer
Complete
Use, don't
sell
Legit core (Sheets·AI

video·Drive, 181 tests) =
keep as your engine. But
ManyChat auto-DM +
stealth-browser stack
violate platform ToS ·
strip those, don't sell.
Saturated market anyway.
Omega (omega_agent)
Complete, not
hardened
·· Month two
Crowded space. Code
artifact only, after ases.
voice_agent_avatar
22+ bugs (7
critical)
· Not yet
1·2 days hardening before
it can even demo.
Reality Check & Strategy · The Numbers

Project
State
Sellable now?
Reality check
god_ai_scripts
Real CV
pipeline +
models
·· Yes, after
repackage
Legit on-device content-
moderation CV system
(violence/gore/NSFW
detect + mask) with OCR,
exports to TFLite/ONNX.
Trained ONNX model
included. Sellable as
model + SDK after
packaging + key

redaction.
api_money_bot
Complete
· Do not sell
ToS-violating key
harvester. NOT the same
project as crypto_trader.
Legal tail you can't carry.
crypto_trader (the real
one)
6-mo paper-
tested,
hardened
·· As study +
source
Distinct from
api_money_bot. Monte-
Carlo 95·98% sim win
rate; live performance
UNVERIFIED (its own risk
doc says so honestly). Sell
as architecture + honest
risk study + full source,
keys redacted. NEVER as
a profit promise.
convergence_framework
Launch-ready
· Yes
Earlier audit's fastest ·499
product. Verify contents,
then list on Instamojo.

3.5 ·· Security · fix BEFORE any sale

Both flagship projects currently ship with your own live secrets inside the zip. Do not upload these anywhere until stripped:

crypto_trader: contains .env , .env1 , and gmgn_keys/gmgn_private.pem . A private key was already exposed once (the risk doc notes it was rotated). Selling as-is leaks your wallet + API credentials.

Reality Check & Strategy · The Numbers

god_ai_scripts:
nsfw_fix.py has your hardcoded Kaggle username + API key
(the comment even claims "safe to keep here" · it is not).
ai_video_monetizer:
client_secret.json contains a real Google OAuth client ID +
secret. Delete it before sharing.
Fix (~30 min): delete/rotate the keys, add .gitignore , ship
a .env.example
instead. This prevents a real disaster.
4. The two honest lanes
Lane 1 · Services (fastest, no tail):
AI video reels for local businesses + indie musicians,
powered by the
ai_video_monetizer pipeline.
Pricing ladder: ·800·1,500 (first jobs) · ·2,000·5,000 ·
·5,000·15,000
(music videos) · ·6,000·15,000/mo retainers.
First rupee: 3·14 days.
Lane 2 · Code artifact (parallel, low effort):
ases sold as-is boilerplate via Instamojo (digital-product
listing + payment
link). ·800·1,500.
Instamojo hosts the file download and collects payment in one
link · no
Gumroad, no India payout friction.
Honest label = the pitch. Zero support obligation.
5. The real bottleneck (now shifted)
Receiving money is SOLVED. Instamojo (full KYC + video KYC) +
UPI + 2 live bank
accounts means the payout side has zero friction. That was
the hardest gate for
someone in your situation, and it's already behind you.
So the bottleneck is no longer can you get paid · it's
demand: getting the first
stranger to say yes. That's an outreach problem, not an
infrastructure problem,
and outreach is cheap, fast, and fully in your control. The
plan is now locked.

6. Next 7 days (draft)
Reality Check & Strategy · The Numbers

Cut a 20·30 sec sample reel that shows character consistency
Write 1 client pitch + 1 musician pitch
Send to 5 real prospects
List ases as-is on Instamojo (digital product + payment link), in parallel
Bank the first rupee · then raise the price
7. Ready to ship (done for you)
Secrets stripped and clean, upload-ready ZIPs prepared for every sellable project.
All three listings are written and copy-paste ready.
Product
Price
Listing
Package status
Convergence
Framework
·499
Listing
· Clean · publish now
Solana Sniper (study + source)
·2,499
Listing
· .env / .env1 /keys removed, .env.example added
SafeScreen (content moderation)
·1,999
Listing
· Kaggle key + kaggle.json removed
ases (code artifact)
·1,500·2,500
·
· Clean source ZIP ready
Omega (code artifact)

·2,000·3,000
·
· Clean source ZIP ready
ai_video_monetizer
not for sale
·
· Cleaned for your own use
(OAuth secret removed)
The clean ZIPs were delivered to you in chat · download and upload straight to
Instamojo. api_money_bot is excluded on purpose: do not sell it.
The single next action
Publish Convergence (·499) today. It's clean, needs zero redaction, and is the
shortest line to your first rupee. Everything else is staged behind it.
Guardrails
Payment channel confirmed: Instamojo (KYC done) + UPI + 2 bank accounts
·
Reality Check & Strategy · The Numbers

No uptime/support promises on unhardened code.
Nothing with legal or ToS risk. Ever, in your situation.
Work in lucid windows. Save after every step. Rest is not failure.
Reality Check & Strategy · The Numbers